Open Critical Mapping Workshop

Hye Ryeon Jang, Ph.D. HJ Political Science Lab Morehouse College

2026-09-24

Introduction

This session introduces how to access and use U.S. Census data to create maps in R. The workshop is designed within an open-source environment, using widely accessible tools and packages.

We will walk through the basic steps of retrieving census data, incorporating spatial information, and generating both static and interactive maps. While this session focuses on foundational skills, these techniques can be readily adapted for use in teaching, research projects, and data-driven analysis.

R Markdown

The file we are currently working in is called an **R Markdown (.Rmd)** file. R Markdown is a reproducible reporting framework that integrates written explanations, R code, and output in one document (e.g., PDF or HTML). As you "knit" the file, the code is executed and the results, such as figures and tables, are automatically generated, making your analysis transparent and easy to share.

Before we import U.S. Census data, we first need to install the necessary package and run a setup function. For this step, you only need to install the package and run the setup function once. You will not need to repeat this process unless you update your R version or reinstall R.

Let's take care of this step before moving forward.

1. Install the **tinytex** package.
2. Run the following code in your R Console: `tinytex::install_tinytex()`
3. To generate a PDF version of this file, click the **Knit** button. The compiled PDF will be saved in your current working directory.

Packages

Let's start by installing/loading packages (as needed) and libraries:

```
# This gray area is called R chunk
# which is a section in an .Rmd file where you write and run R code,
# and the results are automatically included in the final PDF document.

# Install packages as needed
# install.packages(c("tidycensus", "tidyverse", "mapview", "mapgl", "quarto", "sf", "viridis"))
```

```r
# Load necessary libraries
library(tidycensus)
library(tidyverse)
#library(dplyr) if tidyverse is not working
#library(ggplot2) if tidyverse is not working
library(sf)
library(viridis)
library(mapview)
```

U.S. Census API key

This file will support R users with the basics of mapping and spatial analysis with the U.S. Census data using the `tidycensus()` package. Additional information can be found on these slides here.

To access the census data for mapping, please sign up for an API key using this link. You should receive an API key in your email. Place it in the code chunk as indicated below.

```r
census_api_key("YOURKEYHERE")
```

Data

For mapping tasks, we can use our API key to access the American Community Survey (ACS). This is an annual survey that covers topics not available in the decennial US Census data, such as income, education, etc. For our purposes, we will use 5-year estimate data at both the county and census tract level. Remember that the data is *estimated*, and each entry has an associated *margin of error* (moe).

The American Community Survey categorizes data based on variables. For example, the variable for "Black Population" is *B01001B_001*. There are thousands of variables covering all sorts of demographics. You can experiment with looking up variables below. We have also included some commonly measured variables for your convenience.

```r
acs_vars <- load_variables(2023, dataset = "acs5")
#View(acs_vars) # look up variables using the search bar in the top right

# median household income
# B19013_001

# Black population
# B01001B_001

# Population of high school diploma holders
# B15003_017

# Unemployed Population
# B23025_005

# Population of impoverished white people
# B17001A_002

# Population of impoverished Black people
# B17001B_002
```

Gathering data

We will start by gathering the data using the `get_acs()` function.

Now that you know what kind of data you want to look at, let's construct our first API call. This is how we access the data on our computer from the database.

Each API call is made up of four major components:

1. `geography` level: You can control how broad or specific you want the data to be using `geography`. You can organize data according to `state`, `counties`, or census `tract`.
2. `variables`: This tells R what kind of data you are interested in.
3. `state/county`: This specifies where you want to look for data. For example, if your `geography` level is set to `county`, your `state` should specify which state you want to look at. Similarly, if your geography is set to `tract`, you should specify which county you want to examine.
4. `year`: This specifies what time period you want to examine. Keep in mind that data for 2024 onwards may not be updated and may return an error.

Georgia

In this exercise, we will learn how to create a map focusing on Atlanta, Georgia – our neighborhood.

We begin by mapping the entire state of Georgia. Because Georgia covers a large geographic area, we will use **county** as our unit of analysis to provide a clear and interpretable overview.

Among the many available census variables, we will focus on **income** to visualize variation in economic conditions across Georgia. This allows us to see how income levels differ across counties on the map.

```
### Show how to gather data based on variable and geography

ga_income <- get_acs(
  geography = "county",
  variables = "B19013_001",
  state = "GA",
  year = 2023
)

ga_income
```

```
## # A tibble: 159 x 5
##    GEOID NAME                      variable   estimate   moe
##    <chr> <chr>                     <chr>         <dbl> <dbl>
##  1 13001 Appling County, Georgia   B19013_001    43728  6301
##  2 13003 Atkinson County, Georgia  B19013_001    38438  6329
##  3 13005 Bacon County, Georgia     B19013_001    50310  8468
##  4 13007 Baker County, Georgia     B19013_001    44405  9929
##  5 13009 Baldwin County, Georgia   B19013_001    55413  4351
##  6 13011 Banks County, Georgia     B19013_001    68830 10370
##  7 13013 Barrow County, Georgia    B19013_001    77477  3714
##  8 13015 Bartow County, Georgia    B19013_001    79431  3631
##  9 13017 Ben Hill County, Georgia  B19013_001    39081  3479
## 10 13019 Berrien County, Georgia   B19013_001    50414  4735
## # i 149 more rows
```

Fulton County in Georgia

Now, we will load the dataset required to create an income distribution map for Fulton County, Georgia. Since we are working at a smaller scale, we will use **tract** to capture more fine-grained variation.

```
fulton_income <- get_acs(
  geography = "tract",
  variables = "B19013_001",
  state = "GA",
  county = "Fulton",
  year = 2023
)

fulton_income
```

```
## # A tibble: 327 x 5
##    GEOID       NAME                                       variable estimate   moe
##    <chr>       <chr>                                      <chr>       <dbl> <dbl>
##  1 13121000100 Census Tract 1; Fulton County; Georgia    B19013_~   141410 42109
##  2 13121000201 Census Tract 2.01; Fulton County; Georgia B19013_~   139883 35345
##  3 13121000202 Census Tract 2.02; Fulton County; Georgia B19013_~   176332 94519
##  4 13121000400 Census Tract 4; Fulton County; Georgia    B19013_~   156528 31044
##  5 13121000501 Census Tract 5.01; Fulton County; Georgia B19013_~   108940 13526
##  6 13121000502 Census Tract 5.02; Fulton County; Georgia B19013_~   144643 76585
##  7 13121000601 Census Tract 6.01; Fulton County; Georgia B19013_~    78140 23862
##  8 13121000602 Census Tract 6.02; Fulton County; Georgia B19013_~    73340 10103
##  9 13121000700 Census Tract 7; Fulton County; Georgia    B19013_~   147045 49877
## 10 13121001001 Census Tract 10.01; Fulton County; Georg~ B19013_~   102591 11544
## # i 317 more rows
```

Spatial data

Next, we will add spatial data (i.e., the geographic boundaries of each unit with latitude and longitude, such as counties or census tracts) to our income dataset. We do this by enabling simple features (`sf`) using the argument `geometry = TRUE`, which includes the geometry column in the output.

Note that we will use the same code as before, but simply add `geometry = TRUE` as the final argument. We will also update the name of the resulting data frame to reflect that it now contains spatial information.

Georgia Spatial Data

Next, we will retrieve spatial data for Georgia at the county level, which includes the geographic boundaries needed to create our map.

```
ga_income_sf <- get_acs(
  geography = "county",
  variables = "B19013_001",
  state = "GA",
  year = 2023,
  geometry = TRUE
)

ga_income_sf
```

Fulton County spatial data

We will retrieve spatial data for Fulton County at the census tract level to enable fine-grained mapping.

```
fulton_income_sf <- get_acs(
  geography = "tract",
  variables = "B19013_001",
  state = "GA",
  county = "Fulton",
  year = 2023,
  geometry = TRUE
)

fulton_income_sf
```

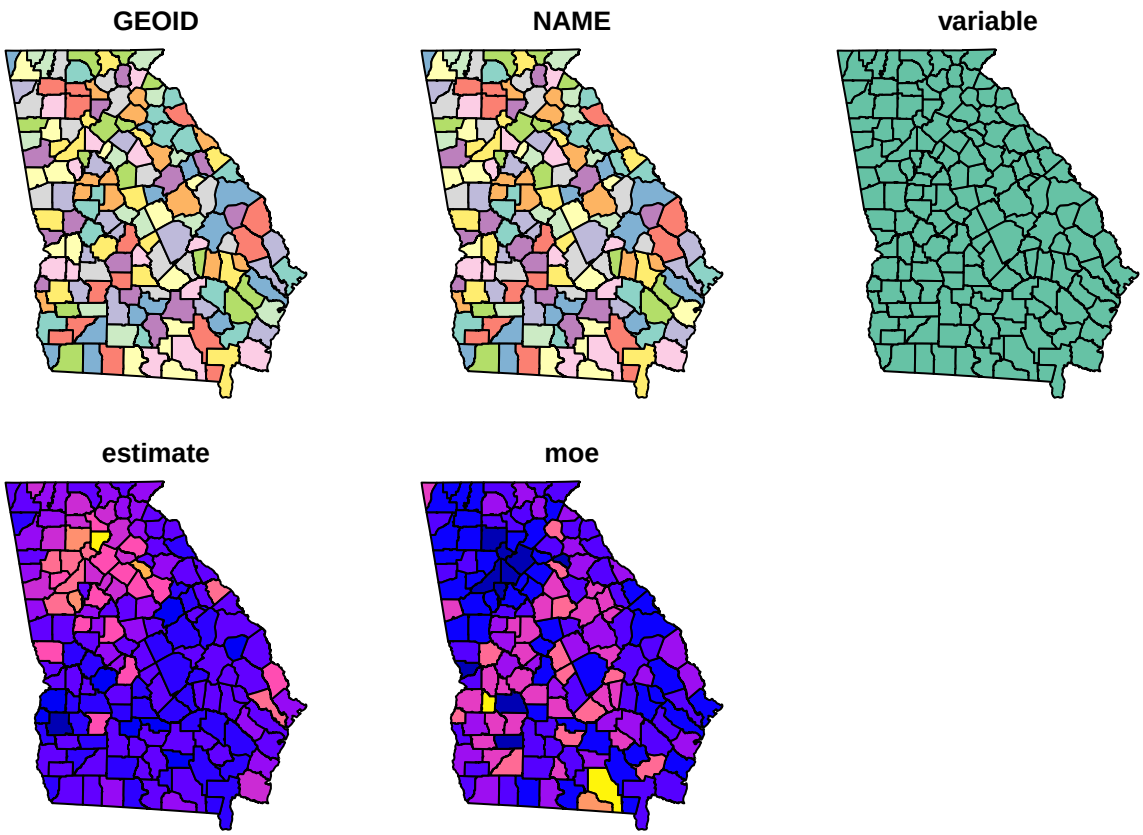
Mapping

Using the dataset we loaded from the U.S. Census Bureau, we can now create a map by applying the `plot()` function to our spatial data frame, which contains both income data and geographic boundaries.

Map: Georgia

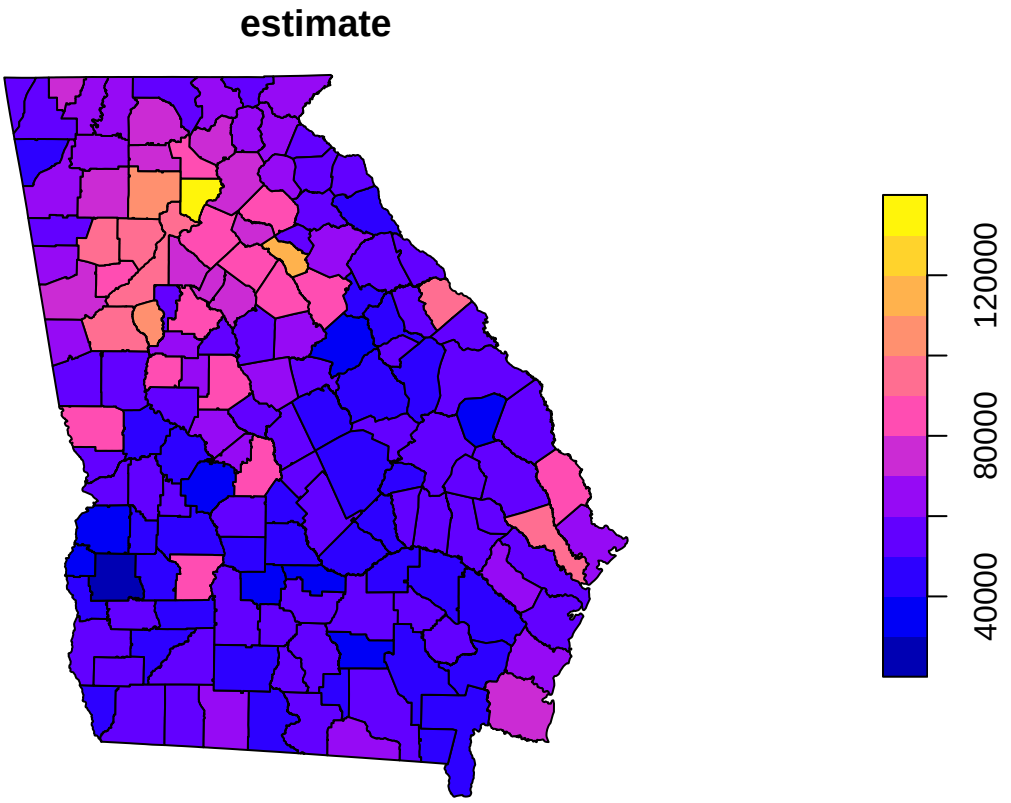
We will use Georgia's income data to create a map. Remember to use the `sf` data frame, as it contains both the income data and the geographic boundaries required for plotting.

```
plot(ga_income_sf)
```



Note that this produces multiple maps - one for each variable in the dataset. To generate a single, meaningful map, we need to specify the *estimate* variable.

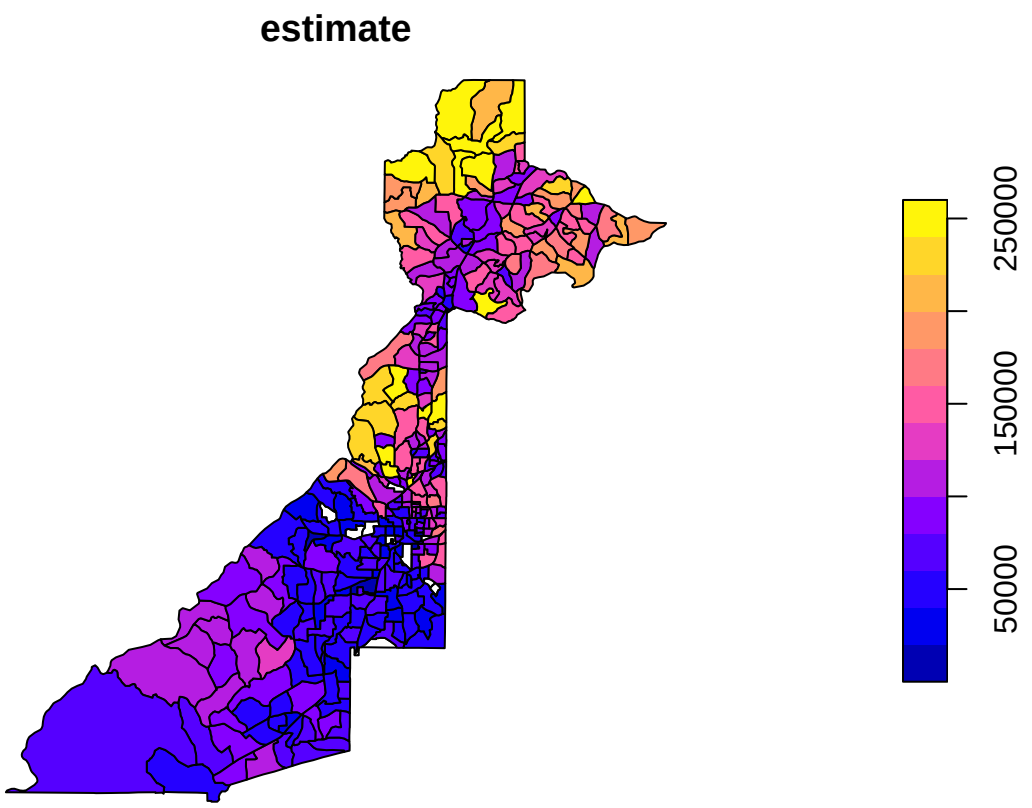
```
plot(ga_income_sf['estimate'])
```



Map: Fulton County

We will repeat the same steps for Fulton County, using tract-level data.

```
plot(fulton_income_sf['estimate'])
```

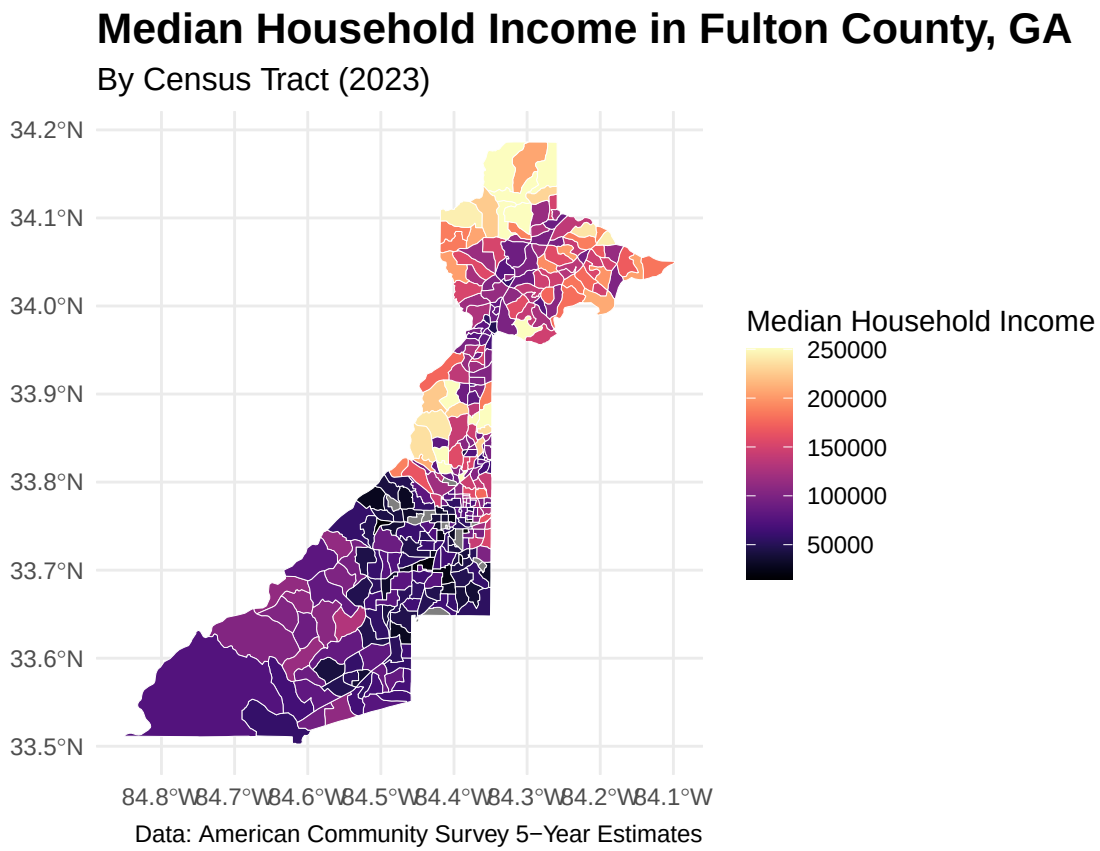


The choice of geographic unit should be guided by your specific research question. From this point onward, we shift our focus to Fulton County to examine more fine-grained, tract-level variation.

Improving Your Map

We can further enhance the visual quality of our maps. In this section, we will use the `viridis` package for improved color scales and switch from `plot()` to `ggplot2` to create more flexible and visually appealing maps.

```
ggplot(fulton_income_sf) +
  geom_sf(aes(fill = estimate), color = "white", size = 0.1) +
  scale_fill_viridis(option = "magma", name = "Median Household Income") +
  labs(
    title = "Median Household Income in Fulton County, GA",
    subtitle = "By Census Tract (2023)",
    caption = "Data: American Community Survey 5-Year Estimates"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    plot.subtitle = element_text(size = 12),
    axis.title = element_blank(),
    legend.position = "right"
  )
```



Interactive Maps

We can also create interactive maps with additional features. Here, we use the `mapview` package to generate an interactive map by passing our `sf` data frame and specifying the *estimate* variable to display.

Please keep in mind that interactive maps cannot be displayed in a PDF file. To view interactive maps, you must knit the document using the HTML output format. To change the output format to HTML, go to the title section (YAML header) at the top of the file and replace `output: pdf_document` with `output: html_document`.

```
mapview(
  fulton_income_sf,
  zcol = "estimate"
)
```

Plotting multiple variables on one map

We can overlay multiple variables on a map. Let's create a map to show Black population density on top of median household income in Atlanta.

Since we already have the Fulton county income data, use `get_acs()` to collect data on Black population.

```
fulton_blackpop <- get_acs(
  geography = "tract",
```

```
  variables = "B01001B_001", # Black population variable
  state = "GA",
  county = "Fulton",
  year = 2023,
  geometry = T
)
```

To make patterns clearer, we convert the data into a dot density map, where each dot represents a fixed number of people.

```
blackpop_dots <- as_dot_density(
  fulton_blackpop,
  value = "estimate",
  values_per_dot = 1000 # adjust this number to
)
```

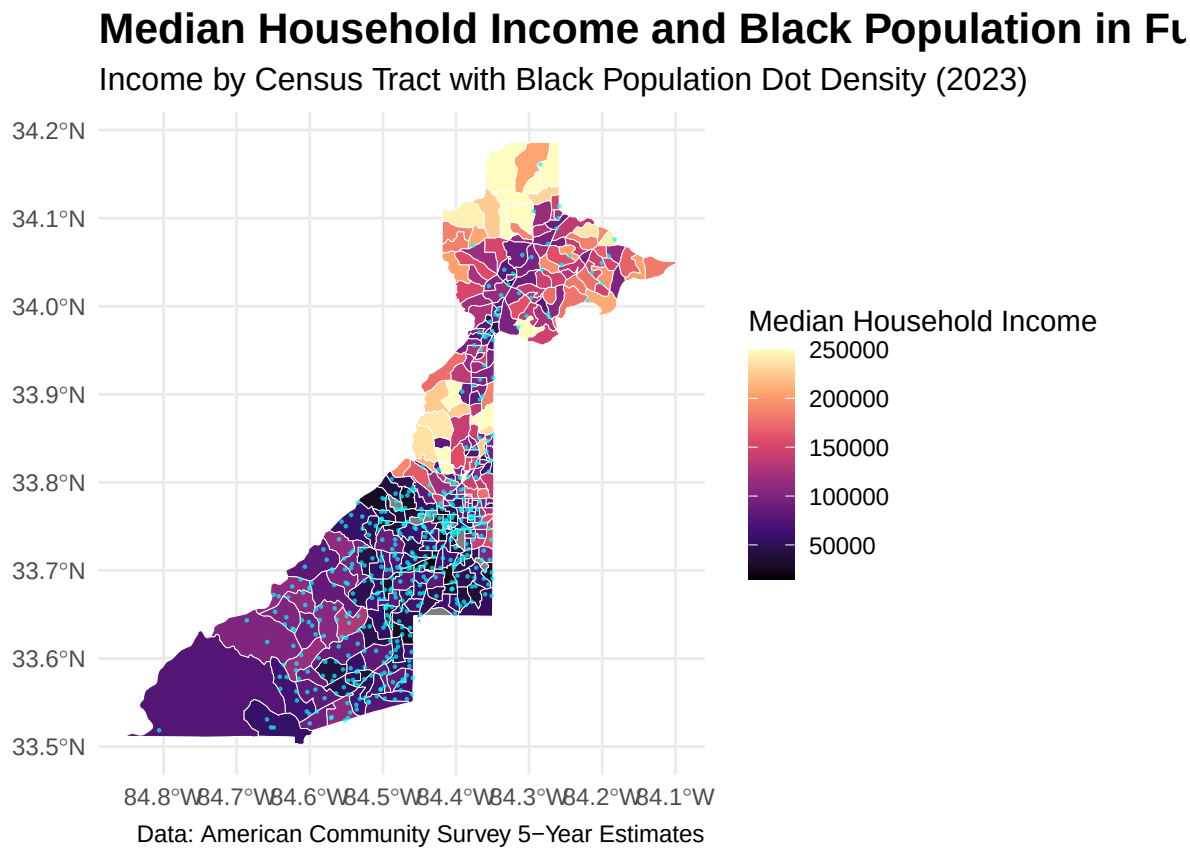
Finally, we can combine both datasets into a single interactive map:

```
mapview(
  fulton_income_sf, # median household income
  zcol = "estimate",
  alpha.regions = 0.75
)+
  mapview(blackpop_dots, cex = 0.01,
          layer.name = "Black population<br>1 dot = 1000 people", # black population
          col.regions = "NA", color = "red")
```

To display both `fulton_income_sf` and `blackpop_dots` in a single static map, you can use the following code with the `ggplot2` package to overlay the two layers.

```
ggplot() +
  geom_sf(
    data = fulton_income_sf,
    aes(fill = estimate),
    color = "white",
    linewidth = 0.1
  ) +
  geom_sf(
    data = blackpop_dots,
    color = "cyan",
    size = 0.15,
    alpha = 0.7,
    inherit.aes = FALSE
  ) +
  scale_fill_viridis_c(
    option = "magma",
    name = "Median Household Income"
  ) +
  labs(
    title = "Median Household Income and Black Population in Fulton County, GA",
    subtitle = "Income by Census Tract with Black Population Dot Density (2023)",
    caption = "Data: American Community Survey 5-Year Estimates"
```

```
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    plot.subtitle = element_text(size = 12),
    axis.title = element_blank(),
    legend.position = "right"
  )
```



Mapping Gentrification

For the workshop, we will use a relatively simple and defensible definition of gentrification (Smith et al., 2024). A census tract is considered eligible to gentrify if its median household income at the beginning of the study period is below the Fulton County median. Among these initially lower-income tracts, we will then examine changes in socioeconomic characteristics and housing costs to identify patterns associated with gentrification.

We will compare ACS data from 2013 and 2019. Both years use 2010 Census tract geography, which allows us to compare census tracts more consistently over time. In contrast, the 2023 ACS uses newer 2020-era tract geography, and changes in census tract boundaries can complicate direct longitudinal comparisons.

For this exercise, we will use three indicators to measure neighborhood change associated with gentrification:

1. Median household income: B19013_001

2. Percentage of adults with a bachelor's degree or higher: B15003_022 through B15003_025, divided by B15003_001
3. Median gross rent: B25064_001

```
# Variables for measuring neighborhood socioeconomic change
gentrification_vars <- c(
  income = "B19013_001",       # Median household income
  educ_total = "B15003_001",   # Population age 25+
  bachelors = "B15003_022",    # Bachelor's degree
  masters = "B15003_023",      # Master's degree
  professional = "B15003_024",# Professional degree
  doctorate = "B15003_025",    # Doctorate degree
  rent = "B25064_001"          # Median gross rent
)
```

We will retrieve the baseline data:

```
fulton_2013 <- get_acs(
  geography = "tract",
  variables = gentrification_vars,
  state = "GA",
  county = "Fulton",
  year = 2013,
  output = "wide"
)
```

And we will also retrieve the later-period data:

```
fulton_2019 <- get_acs(
  geography = "tract",
  variables = gentrification_vars,
  state = "GA",
  county = "Fulton",
  year = 2019,
  output = "wide",
  geometry = TRUE
)
```

```
##   |                                                                              |
```

```
fulton_2013 <- fulton_2013 %>%
  mutate(
    college_pct_2013 =
      100 * (bachelorsE + mastersE + professionalE + doctorateE) /
      educ_totalE
  ) %>%
  select(
```

```
    GEOID,
    income_2013 = incomeE,
    rent_2013 = rentE,
    college_pct_2013
  )
```

And we will do the same thing for 2019:

```
fulton_2019 <- fulton_2019 %>%
  mutate(
    college_pct_2019 =
      100 * (bachelorsE + mastersE + professionalE + doctorateE) /
      educ_totalE
  ) %>%
  select(
    GEOID,
    geometry,
    income_2019 = incomeE,
    rent_2019 = rentE,
    college_pct_2019
  )
```

Then we will combine the two years:

```
fulton_change <- fulton_2019 %>%
  left_join(fulton_2013, by = "GEOID")
```

Now let's calculate neighborhood change:

```
fulton_change <- fulton_change %>%
  mutate(
    income_change =
      100 * (income_2019 - income_2013) / income_2013,

    rent_change =
      100 * (rent_2019 - rent_2013) / rent_2013,

    college_change =
      college_pct_2019 - college_pct_2013
  )
```

Based on the neighborhood changes in income, rent, and education, we will calculate Fulton County benchmarks:

```
fulton_income_median_2013 <-
  median(fulton_change$income_2013, na.rm = TRUE)

median_income_change <-
  median(fulton_change$income_change, na.rm = TRUE)

median_rent_change <-
  median(fulton_change$rent_change, na.rm = TRUE)
```

```
median_college_change <-
  median(fulton_change$college_change, na.rm = TRUE)
```

The function `na.rm = TRUE` removes missing values (NA) before doing the calculation.

Finally we can construct a simple gentrification indicator.

```
fulton_change <- fulton_change %>%
  mutate(

    # Was the tract relatively low-income at the beginning?
    gentrifiable =
      income_2013 < fulton_income_median_2013,

    # Socioeconomic upgrading
    education_growth =
      college_change > median_college_change,

    # Increasing housing costs
    housing_growth =
      rent_change > median_rent_change,

    # Income growth
    income_growth =
      income_change > median_income_change
  )
```

For the actual level of gentrification, I would create a 0-3 indicator rather than simply saying "gentrified/not gentrified":

```
fulton_change <- fulton_change %>%
  mutate(
    gentrification_score = case_when(
      !gentrifiable ~ NA_real_,
      TRUE ~ education_growth +
             housing_growth +
             income_growth
    )
  )
```

`case_when()` lets us assign different values depending on different conditions. It works somewhat like an `if/else` statement. `!gentrifiable ~ NA_real_,` indicates if the tract was not initially below the county median income, do not calculate a gentrification score for it. `NA_real_` specifically means a missing numeric value. `TRUE ~` means for everything else that did not meet the earlier condition. If a tract meets each condition, R treats logical values as numbers when we add them: TRUE = 1 and FALSE = 0. Thus, the maximum value of `gentrification_score` is 3 and the minimum value is 0.

`case_when()` allows us to assign different values based on different conditions, much like an if/else statement. The line `!gentrifiable ~ NA_real_` means that if a census tract was not initially below the county median household income, we do not calculate a gentrification score for that tract. `NA_real_` represents a missing numeric value. The line beginning with `TRUE ~` applies to all remaining tracts that did not meet the previous condition. For those tracts, R adds the three logical indicators. When logical values are used in arithmetic, R treats `TRUE` as 1 and `FALSE` as 0. Therefore, the gentrification_score ranges from 0 to 3, depending on how many of the three indicators are present.

So among initially lower-income census tracts:

- 0 = no indicators
- 1 = one indicator
- 2 = two indicators
- 3 = all three indicators

We can call this indicator a **Gentrification Indicator Score**. It is not an objective or universal measure of gentrification. Definitions of gentrification do vary substantially and different operational definitions can produce different classifications (Smith et al., 2024).

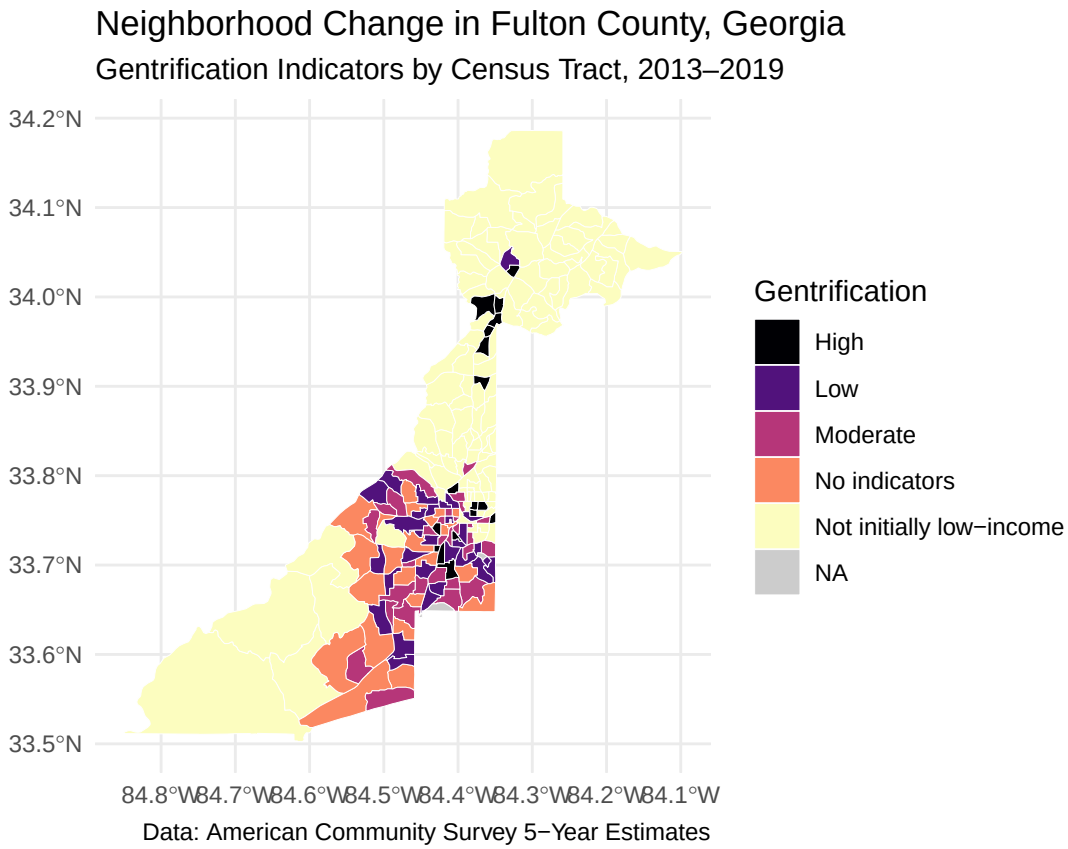
Now let's turn it into categories:

```
fulton_change <- fulton_change %>%
  mutate(
    gentrification_level = case_when(
      !gentrifiable ~ "Not initially low-income",
      gentrification_score == 0 ~ "No indicators",
      gentrification_score == 1 ~ "Low",
      gentrification_score == 2 ~ "Moderate",
      gentrification_score == 3 ~ "High",
      TRUE ~ NA_character_
    )
  )
```

`TRUE ~ NA_character_` indicates for all remaining cases that did not match an earlier condition, assign a missing character value.

And finally, this fits almost perfectly with the mapping code we already created in the workshop:

```
ggplot(fulton_change) +
  geom_sf(
    aes(fill = gentrification_level),
    color = "white",
    linewidth = 0.1
  ) +
  scale_fill_viridis_d(
    option = "magma",
    na.value = "grey80",
    name = "Gentrification"
  ) +
  labs(
    title = "Neighborhood Change in Fulton County, Georgia",
    subtitle = "Gentrification Indicators by Census Tract, 2013-2019",
    caption = "Data: American Community Survey 5-Year Estimates"
  ) +
  theme_minimal() +
  theme(
    axis.title = element_blank(),
    legend.position = "right"
  )
```

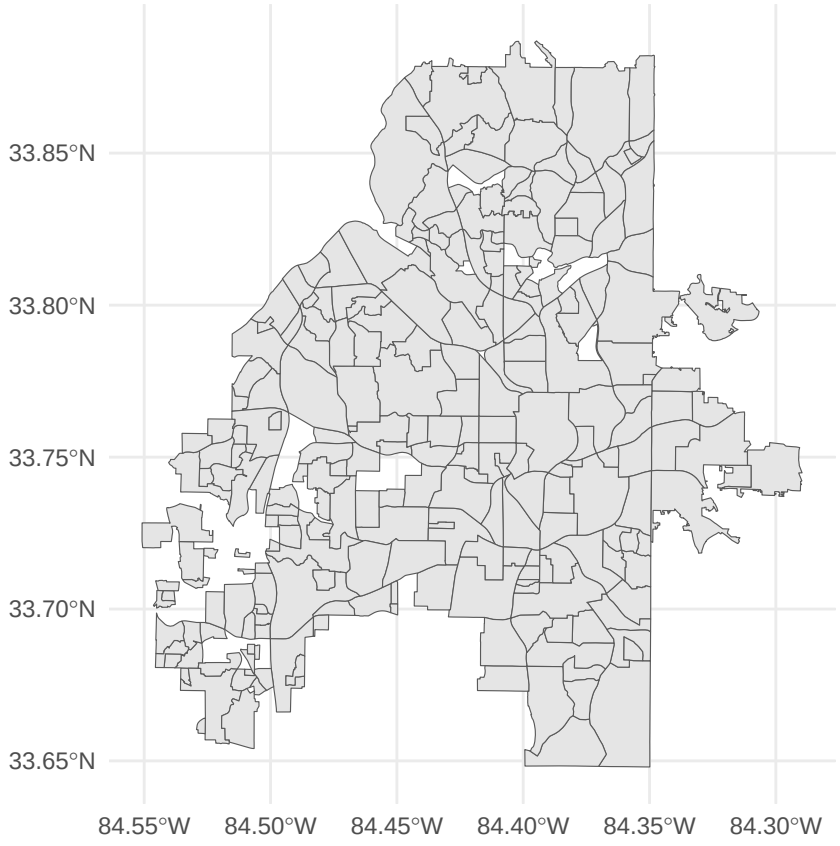


Mapping Neighborhood Level Data

```
atlanta <- st_read("Official_Neighborhoods_-_Open_Data.shp")
```

```
## Reading layer 'Official_Neighborhoods_-_Open_Data' from data source
##   'C:\Users\jjang\Dropbox\[Research]Voting\professional development workshop\mapping_with_census\Offi
##   using driver 'ESRI Shapefile'
## Simple feature collection with 248 features and 20 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -9412158 ymin: 3981633 xmax: -9383078 ymax: 4013621
## Projected CRS: WGS 84 / Pseudo-Mercator
```

```
ggplot(atlanta) +
  geom_sf() +
  theme_minimal()
```




```
ggplot() +

  # Base layer: income
  geom_sf(
    data = fulton_income_sf,
    aes(fill = estimate),
    color = "white",
    linewidth = 0.1
  ) +

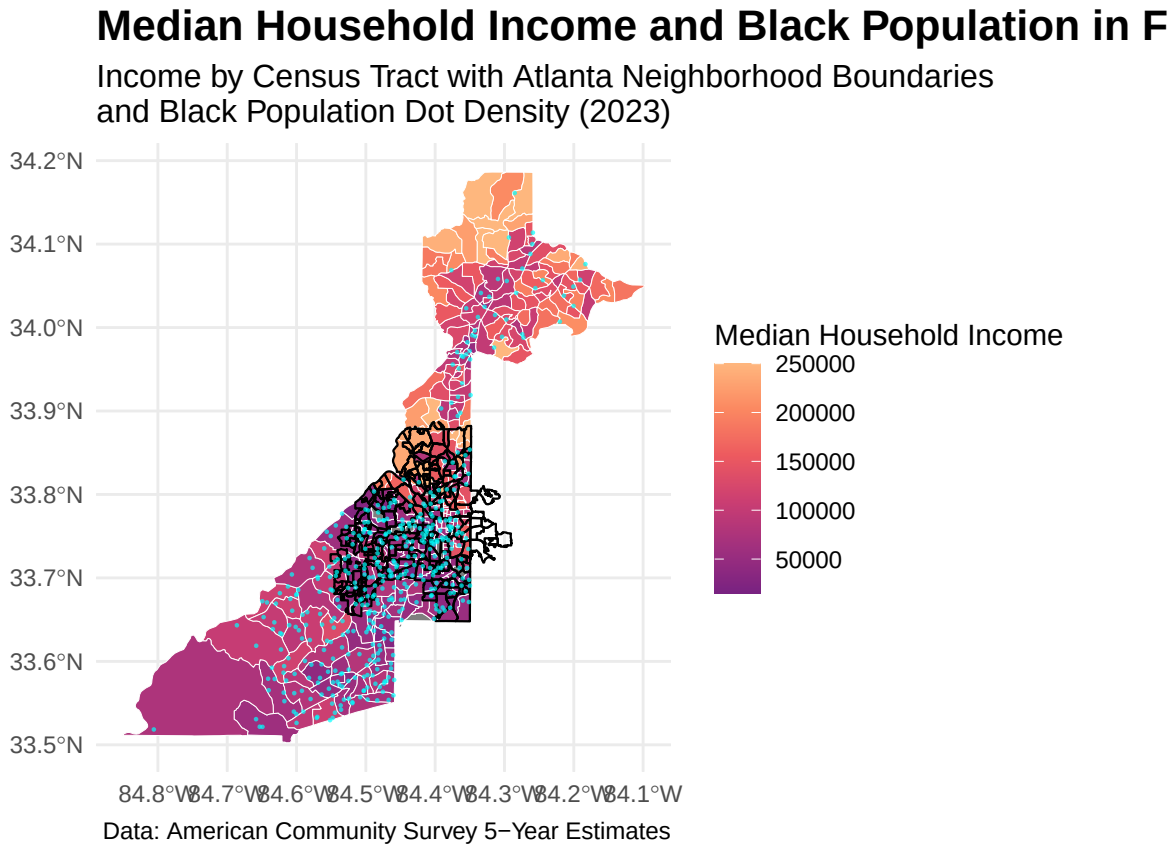
  # Overlay: Atlanta neighborhood boundaries
  geom_sf(
    data = atlanta,
    fill = NA,
    color = "black",
    linewidth = 0.4
  ) +

  # Overlay: dot density
  geom_sf(
    data = blackpop_dots,
    color = "cyan",
    size = 0.15,
    alpha = 0.7,
    inherit.aes = FALSE
  ) +
```

```
  scale_fill_viridis_c(
    option = "magma",
    begin = 0.35,
    end = 0.85,
    name = "Median Household Income"
  ) +

  labs(
    title = "Median Household Income and Black Population in Fulton County, GA",
    subtitle = "Income by Census Tract with Atlanta Neighborhood Boundaries
and Black Population Dot Density (2023)",
    caption = "Data: American Community Survey 5-Year Estimates"
  ) +

  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    plot.subtitle = element_text(size = 12),
    axis.title = element_blank(),
    legend.position = "right"
  )
```



Mapping Data outside the US

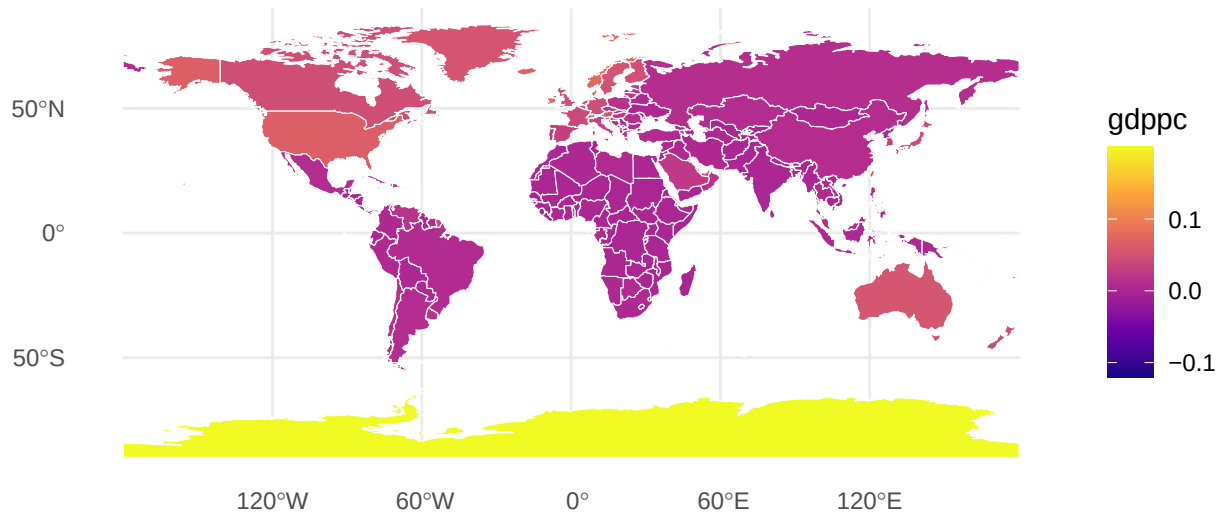
R is an open-ended software. You can achieve anything with the right dataset, such as global maps.

```
#install.packages(c("rnaturalearth", "rnaturalearthdata"))
library(rnaturalearth)
library(rnaturalearthdata)

world <- ne_countries(scale = "medium",
                      #continent = "africa",
                      returnclass = "sf")

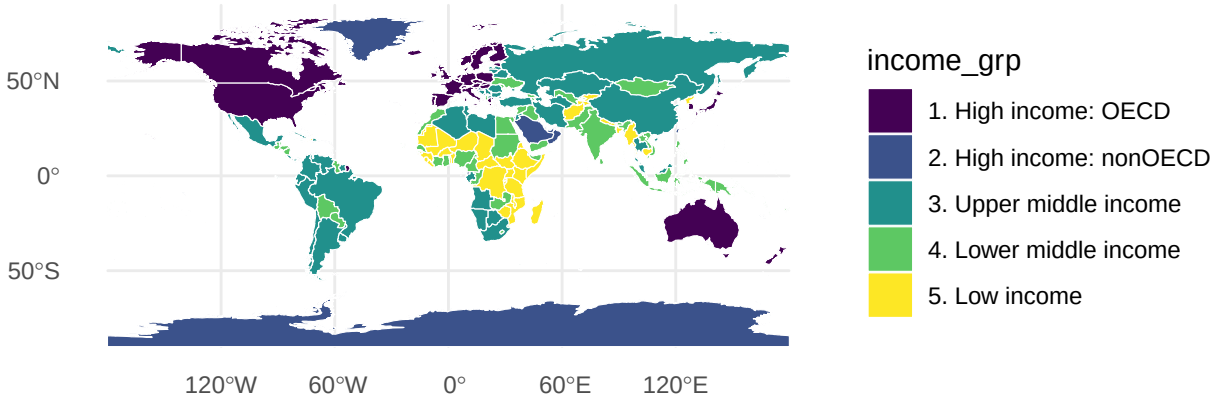
world$gdppc <- world$gdp_md/world$pop_est
head(world)
```

```
ggplot(world)+
  geom_sf(aes(fill = gdppc), color = "white", linewidth = 0.2) +
  scale_fill_viridis_c(option = "plasma", na.value = "grey90")+
  theme_minimal()
```



We can also create a map of income bracket.

```
ggplot(world)+
  geom_sf(aes(fill = income_grp), color = "white", linewidth = 0.2) +
  scale_fill_viridis_d()+
  theme_minimal()
```



Conclusion

Thank you for attending today's workshop. If you have any questions, please feel free to reach out to Hye Ryeon Jang, Ph.D..

We gratefully acknowledge the support of the ORCA Catalytic Award, the Department of Political Science at Morehouse College, and the Humanities, Social Sciences, Media, and Arts (HSSMA) Division for making this workshop possible.

This workshop was developed in collaboration with the Quantitative History Workshop (PI: Nathan Alexander, Ph.D.) at Howard University.

Special thanks to Myles Ndiritu and undergraduate TAs from HJ Political Science Lab!

Thank you again for your participation!